

Sprint 2 Requirements Artifacts

Linux Kernel Security Monitor (LKSM)

Team Number: Group 32

Team Members: Brett Balquist, Kaden Huber, Jamie King,
Dustin Le, Max Biundo, Hart Nurnberg

Sprint: 2 (Weeks 3-4)

Total Story Points: 34

Req ID	Description	SP
5	Hook to detect hiding_stat_init() behavior	8
6	Hook to detect hooking_audit_init() behavior	5
7	Python script parses events to JSON logs	3
8	Hook file-open events on sensitive paths	8
9	Hook to detect taskstats_hook_init() behavior	5
10	Create Makefile and automation for hooks	5

Requirement 5: Hook to Detect hiding_stat_init() Behavior

Description:

Hook to detect hiding_stat_init() behavior: monitor stat-related syscalls for rootkit detection

Story Points: 8 | Priority: 5

Acceptance Criteria:

5.1 Syscall Identification and Hooking

- Identify stat-family syscalls targeted by rootkits (stat, lstat, fstat, newfstatat)
- Implement kprobe hooks on __do_sys_newstat, __do_sys_newlstat, or equivalent kernel functions
- Capture syscall arguments including pathname and file descriptor

5.2 Behavioral Analysis

- Detect patterns consistent with file/process hiding (ENOENT returned for existing files)
- Monitor for stat calls on /proc entries that return inconsistent results
- Track frequency of stat failures per process to identify anomalous behavior

5.3 Event Logging

- Log events with: timestamp, PID, process name, target path, return value
- Flag suspicious patterns with severity level in log output
- Export events through established kernel-to-user communication channel

5.4 Testing and Validation

- Module loads/unloads without kernel errors
- Normal stat operations are logged correctly
- Simulated hiding behavior triggers appropriate alerts

Deliverables:

- hook_stat_hiding.c - Kernel module source for stat hooks
- Documentation of detection methodology
- Test script demonstrating detection capability

Requirement 6: Hook to Detect hooking_audit_init() Behavior

Description:

Hook to detect hooking_audit_init() behavior: monitor audit subsystem modifications

Story Points: 5 | Priority: 6

Acceptance Criteria:

6.1 Audit Subsystem Monitoring

- Identify kernel audit interfaces commonly targeted (audit_log, audit_log_start)
- Implement hooks to monitor access to audit-related kernel functions
- Detect attempts to disable or bypass audit logging

6.2 Integrity Verification

- Monitor audit configuration changes via auditctl or direct manipulation
- Track audit rule additions, deletions, and modifications
- Detect audit daemon (auditd) termination or interference

6.3 Event Capture and Reporting

- Log all audit subsystem interactions with full context
- Generate alerts for unauthorized audit modifications
- Include process credentials (UID, EUID) in event data

Deliverables:

- hook_audit_monitor.c - Kernel module for audit subsystem monitoring
- Alert criteria documentation
- Test cases for audit tampering detection

Requirement 7: Python Script for Event Parsing and JSON Logging

Description:

Python script parses process events and writes structured JSON log files

Story Points: 3 | Priority: 7

Acceptance Criteria:

7.1 Event Parsing

- Read raw events from kernel communication channel (/proc/lksm or netlink)
- Parse binary or text event format into Python data structures
- Handle malformed or incomplete events gracefully

7.2 JSON Structure

- Define consistent JSON schema for all event types
- Include fields: event_type, timestamp, pid, ppid, uid, comm, details
- Support extensible detail fields for hook-specific data

7.3 File Management

- Write JSON logs to configurable output directory
- Implement log rotation by size or time interval
- Ensure atomic writes to prevent log corruption

7.4 Performance

- Process events in real-time without significant lag
- Buffer events appropriately to handle bursts
- Minimal CPU and memory footprint during operation

Deliverables:

- event_parser.py - Main parsing script
- json_schema.json - Event schema definition
- Sample output logs demonstrating format

Requirement 8: Hook File-Open Events on Sensitive Paths

Description:

Hook file-open events on sensitive paths (/etc/passwd, /etc/shadow, SSH keys) in the LKM

Story Points: 8 | Priority: 8

Acceptance Criteria:

8.1 File Open Interception

- Hook `do_sys_open`, `do_sys_openat`, or `filp_open` kernel functions
- Capture file path, open flags (read/write/append), and mode
- Resolve relative paths to absolute paths for accurate matching

8.2 Sensitive Path Monitoring

- Monitor `/etc/passwd` and `/etc/shadow` access
- Monitor SSH key directories (`~/.ssh/`, `/etc/ssh/`)
- Monitor `/etc/sudoers` and sudo configuration files
- Support configurable path list for extensibility

8.3 Access Context Capture

- Log accessing process PID, UID, GID, and executable path
- Record parent process information for chain analysis
- Capture timestamp with nanosecond precision

8.4 Alerting Logic

- Flag write attempts to read-only sensitive files
- Alert on access from unexpected processes (non-system)
- Differentiate between normal system operations and suspicious access

Deliverables:

- `hook_file_access.c` - Kernel module for file monitoring
- `sensitive_paths.conf` - Configurable path list
- Test results showing detection of sensitive file access

Requirement 9: Hook to Detect taskstats_hook_init() Behavior

Description:

Hook to detect taskstats_hook_init() behavior: monitor task statistics interface tampering

Story Points: 5 | Priority: 9

Acceptance Criteria:

9.1 Taskstats Interface Monitoring

- Identify taskstats netlink interface and related kernel functions
- Hook taskstats_exit and related process accounting functions
- Monitor for modifications to task statistics reporting

9.2 Tampering Detection

- Detect attempts to hide process resource usage
- Monitor for inconsistencies between taskstats and /proc data
- Track unusual patterns in task accounting data

9.3 Integration

- Export events through common communication channel
- Follow established event format for consistency
- Coordinate with other hooks to avoid conflicts

Deliverables:

- hook_taskstats.c - Kernel module for taskstats monitoring
- Integration documentation with other hooks
- Test cases for tampering scenarios

Requirement 10: Create Makefile and Automation for Hooks

Description:

Create Makefile and automation scripts to compile, load, and run all hooks systematically

Story Points: 5 | Priority: 10

Acceptance Criteria:

10.1 Makefile Structure

- Single top-level Makefile builds all kernel modules
- Support individual module targets (make hook_stat, make hook_audit, etc.)
- Include 'make clean' target to remove all build artifacts
- Include 'make all' target to build entire project

10.2 Module Management Scripts

- load_all.sh - Script to load all hooks in correct order
- unload_all.sh - Script to safely unload all hooks
- status.sh - Script to check loaded module status
- Handle dependencies between modules if any exist

10.3 Integration with Python Tools

- run_monitor.sh - Script to start kernel modules and Python parser together
- stop_monitor.sh - Script for graceful shutdown of entire system
- Proper signal handling for clean termination

10.4 Error Handling and Logging

- Scripts check for root privileges before module operations
- Meaningful error messages for common failure scenarios
- Log script operations for debugging purposes

Deliverables:

- Makefile - Top-level build configuration
- scripts/ directory with all automation scripts
- USAGE.md documenting all make targets and scripts